Title: Using SQLite and Python to Develop an Ethical Data and Analytical Understanding for a Simulated Retail Database Design

Abstract:

Through an emphasis on ethical data generation and effective database management, this report describes the construction of a simulated retail database using SQLite and Python. Without using actual personal data, we created a fictitious dataset that accurately depicts a retail environment, including items, customers, and orders, by utilizing the faker library and NumPy. Separate tables for each entity and proper use of foreign keys to preserve data integrity are features of the scalable and normalized database schema. We illustrate how the database can facilitate intricate analyses and business decision-making procedures using sample SQL queries. This project provides insights into customer behaviour and sales while highlighting the significance of ethical practices in data generation and demonstrating a useful approach to database design and utilization in a retail context.

Methodology for Generating Data for Retail Store Database:

I used Python, its robust data manipulation library pandas, and the Faker library to create fictitious but realistic data in order to build a synthetic retail store database. This method was selected to build an extensive dataset that includes orders, customers, and products, simulating a real-world retail setting. In order to produce a rich dataset for different analyses, the methodology makes sure that a variety of data types, such as nominal, ordinal, interval, and ratio, are included across the tables.

```
import sqlite3
import pandas as pd
import numpy as np
from faker import Faker
```

```
def create_database():
    """
    Create a SQLite database and tables for storing products, customers, and orders data.
    """
    conn = sqlite3.connect('Retail_store_data.db')
    cursor = conn.cursor()

    # Create Products table
    cursor.execute('''
    CREATE TABLE IF NOT EXISTS Products (
        ProductID INTEGER PRIMARY KEY,
        Category TEXT,
        Price REAL,
        StockLevel INTEGER,
        Rating INTEGER,
        SalesVolume INTEGER,
        DiscountRate REAL,
        DiscountedPrice REAL
    )
    ''')
```

A section of Python code for making a SQLite database called "Retail_store_data.db" is shown in the image. Product-related data is defined in a table named "**Products**" within this database. With the product ID designated as the primary key, the table has fields for the product ID, category, price, stock level, rating, sales volume, discount rate, and discounted price.

```
# Create Customers table
cursor.execute('''
CREATE TABLE IF NOT EXISTS Customers (
    CustomerID INTEGER PRIMARY KEY,
    Name TEXT,
    Age INTEGER,
    Gender TEXT,
    MembershipLevel TEXT,
    SignUpDate DATE,
    LastPurchaseDate DATE,
    TotalSpent REAL
)
''')

# Create Orders table
cursor.execute('''
CREATE TABLE IF NOT EXISTS Orders (
    OrderID INTEGER PRIMARY KEY,
    CustomerID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER,
    OrderDate DATE,
    ShippingMethod TEXT,
    OrderTotal REAL,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
)
''')

return conn
```

The **'Customers' table** contains fields like CustomerID (primary key), Name, Age, Gender, MembershipLevel, SignUpDate, LastPurchaseDate, and TotalSpent and is intended to hold customer-related data. Customers' membership information, spending patterns, and demographic data can all be stored and managed with this configuration, which is useful for targeted marketing and customer relationship management.

whereas the **'Orders' table** is meant to record specifics about orders placed by customers. The orderID, which is the primary key, is followed by the following: OrderTotal, OrderDate, OrderDate, Quantity, CustomerID, and ProductID. Additionally, it defines two foreign keys, ProductID and CustomerID, which point to the respective primary keys in the "Products" and "Customers" tables. In order to enable thorough sales analysis and inventory management, this relational setup is essential for tracking which customers ordered which products, the order quantities, and the financials of each transaction.

Realistic and Variable: The data was created to represent scenarios that are realistic in a retail setting. A large range, but within acceptable bounds, was intended for prices and stock

levels, as one might anticipate in a retail environment. The data could now be made more variable by simulating sales volumes and consumer spending habits thanks to the use of NumPy's random functions.

```python
def insert_data(conn, table_name, dataframe):
    """
    Insert data into the specified SQLite table from a pandas DataFrame.

    Parameters:
        conn: SQLite connection object.
        table_name (str): Name of the table where data will be inserted.
        dataframe (DataFrame): pandas DataFrame containing the data to insert.
    """
    dataframe.to_sql(table_name, conn, if_exists='append', index=False)

def generate_product_data(n=1000):
    fake = Faker()
    categories = [
        'Groceries', 'Electronics', 'Clothing and Apparel', 'Home and Garden',
        'Health and Beauty', 'Toys and Games', 'Automotive', 'Sports and Outdoors',
        'Pharmacy and Health Services', 'Furniture', 'Office Supplies', 'Pet Supplies',
        'Baby and Toddler', 'Jewelry and Watches', 'Party Supplies'
    ]

    np.random.seed(42)
    product_data = pd.DataFrame({
        'ProductID': range(1, n + 1),
        'Category': np.random.choice(categories, n),
        'Price': np.random.normal(100, 50, n).round(2),
        'StockLevel': np.random.randint(0, 1000, n),
        'Rating': np.random.choice([1, 2, 3, 4, 5], n),
        'SalesVolume': np.random.randint(0, 5000, n),
        'DiscountRate': np.round(np.random.uniform(0.05, 0.2, n), 2)
    })

    product_data['DiscountedPrice'] = product_data['Price'] * (1 - product_data['DiscountRate'])

    return product_data
```

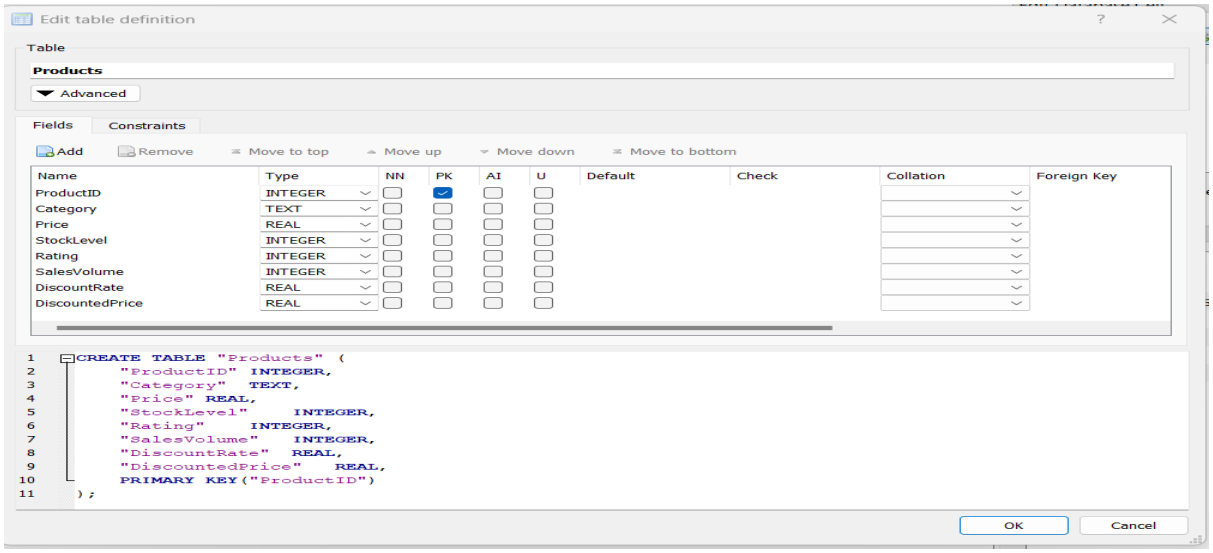
The provided code snippet defines two functions: generate_product_data, which creates a DataFrame of synthetic product data, and insert_data, which inserts data from a pandas DataFrame into a specified SQLite table. With prices and discounts applied to determine the discounted price, the generate_product_data function generates data for 1000 products with attributes like category, price, stock level, rating, sales volume, and discount rate.

```python
def generate_customers_data(n=1000):
    fake = Faker()
    np.random.seed(42)
    customers_data = pd.DataFrame({
        'CustomerID': range(1, n + 1),
        'Name': [fake.name() for _ in range(n)],
        'Age': np.random.randint(18, 70, n),
        'Gender': np.random.choice(['Male', 'Female', 'Other'], n),
        'MembershipLevel': np.random.choice(['Bronze', 'Silver', 'Gold', 'Platinum'], n, p=[0.4, 0.3, 0.2, 0.1]),
        'SignUpDate': pd.date_range(start='2015-01-01', periods=n, freq='24H').strftime('%Y-%m-%d'),
        'LastPurchaseDate': pd.date_range(start='2023-01-01', periods=n, freq='24H').strftime('%Y-%m-%d'),
        'TotalSpent': np.random.normal(1000, 500, n).round(2)
    })

    return customers_data
def generate_orders_data(customers_data, product_data, n=1000):
    np.random.seed(42)
    orders_data = pd.DataFrame({
        'OrderID': range(1, n + 1),
        'CustomerID': np.random.choice(customers_data['CustomerID'], n),
        'ProductID': np.random.choice(product_data['ProductID'], n),
        'Quantity': np.random.randint(1, 5, n),
        'OrderDate': pd.date_range(start='2023-01-01', periods=n, freq='24H').strftime('%Y-%m-%d'),
        'ShippingMethod': np.random.choice(['Standard', 'Expedited', 'Two-Day', 'Same-Day'], n),
        'OrderTotal': (np.random.normal(200, 100, n) * np.random.randint(1, 5, n)).round(2)
    })
    return orders_data
if __name__ == "__main__":
    conn = create_database()

    product_data = generate_product_data()
    customers_data = generate_customers_data()
    orders_data = generate_orders_data(customers_data, product_data)

    # Insert the data into the database
    insert_data(conn, 'Products', product_data)
    insert_data(conn, 'Customers', customers_data)
    insert_data(conn, 'Orders', orders_data)

    conn.commit()
    conn.close()
```

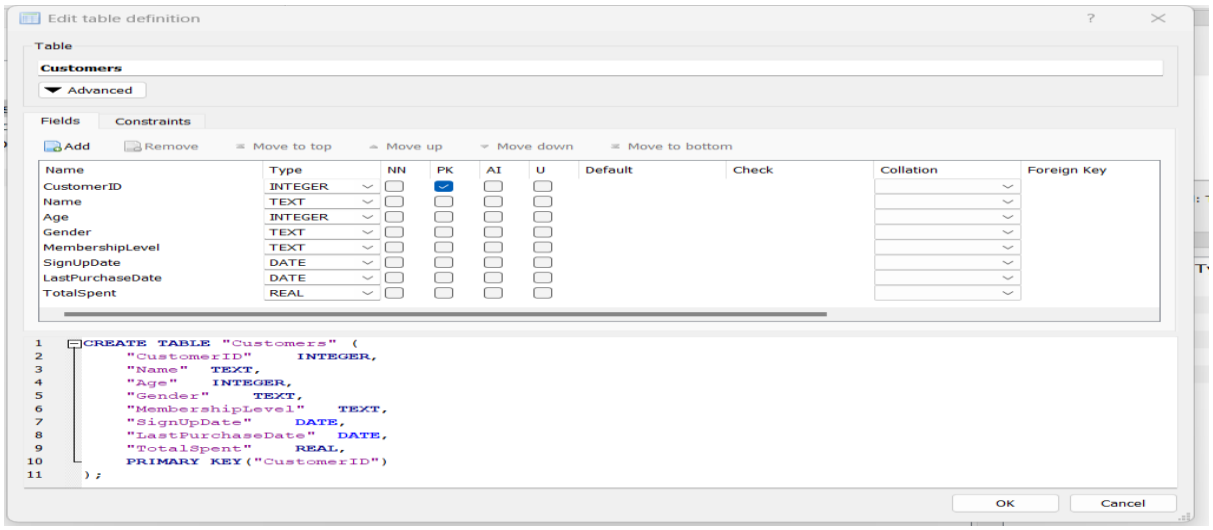
There are two functions in this code snippet that create fake data for a retail database. A DataFrame containing the following customer data is created by the generate_customers_data function: ID, name, age, gender, membership level, sign-up and last purchase dates, and total spent. Order details, which link customers and products by their IDs, are produced by the generate orders data function. This includes the quantity, order date, shipping method, and order total. For reproducibility, the data is randomized using a fixed seed. Then, using the insert_data function, the generated data is inserted into a SQLite database.

Tables and Relationships in the Database Schema:

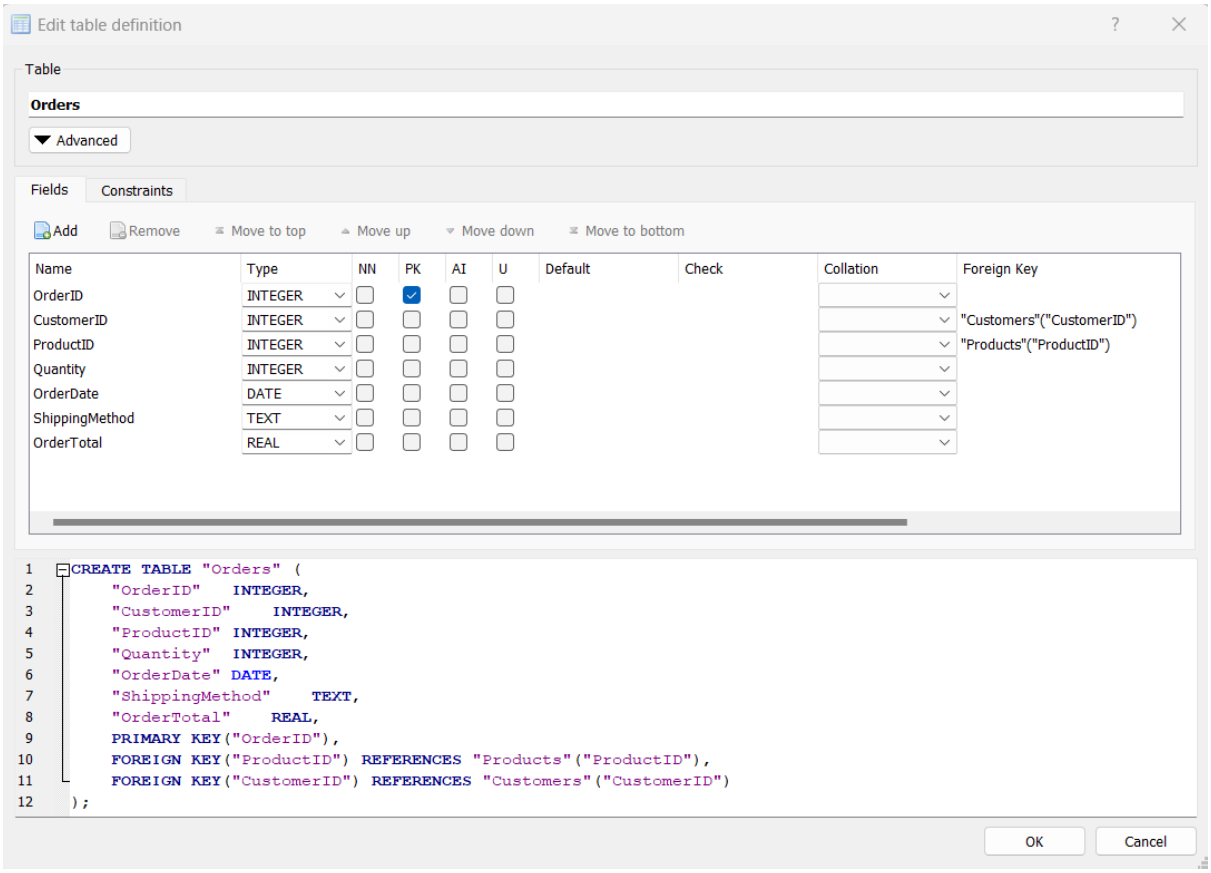
Products Table: This table helps with inventory management and sales analytics by organizing data about products into fields for product ID (primary key), category, price, stock level, rating, sales volume, discount rate, and discounted price.



Customers Table: This table allows for customer behavior analysis and segmentation by providing customer-specific information such as a unique customer ID (primary key), name, age, gender, membership level, sign-up date, last purchase date, and total amount spent.



Orders Table: Contains fields for order ID (primary key), customer ID, product ID, quantity, order date, shipping method, and order total. Foreign keys connect this table to the 'Customers' and 'Products' tables, which are essential for comprehending order fulfillment procedures and purchasing trends.



Foreign and Composite Keys: Relational integrity is established and sophisticated cross-table queries are made possible through the use of foreign keys, such as ProductID in orders linked to products and CustomerID in orders linked to customers. This schema does not use composite keys; instead, primary and foreign keys were carefully selected to guarantee data integrity and facilitate effective querying.

Rationale for Independent Tables and Ethical Conversation

Reasoning behind Different Tables :

The data is divided into the "Products," "Customers," and "Orders" tables in accordance with database normalization principles, which are designed to lessen redundant data and enhance data integrity. Both scalability and effective data management are made possible by this modular structure. For example:

Products Table: By consolidating product data into a single table, update anomalies are reduced and data consistency is preserved. Updates to product details, such as price or stock

adjustments, are centralized.
Customers Table: Keeping a distinct customers table makes it possible to store customer data securely and manage customer profiles more easily. Analyzing consumer demographics and purchase patterns is made easier without the complication of transactional data.
Orders Table: Serving as a link between "Products" and "Customers," this table captures the many-to-many relationships that are typical of retail transactions, in which a customer may buy several products, and a product may be purchased by numerous customers. Without sacrificing the organization of product or customer data, it makes the analysis of sales data and order history easier.

Discussion of Ethics
By avoiding the use of actual personal data and instead creating fake data using the Faker library, the database design takes privacy and data protection seriously. This strategy is in line with moral principles and privacy regulations like GDPR, which stress how crucial it is to preserve personal privacy. The database eliminates the risk of privacy violations by using representative but fictitious data that cannot be linked to specific individuals. This enables robust data analysis and application development.

Example Queries:

Queries for Selections:

```
-- Select product details
SELECT ProductID, Category, Price FROM Products WHERE Category = 'Electronics';
```

	ProductID	Category	Price
4	67	Electronics	108.91
5	72	Electronics	60.98
6	86	Electronics	110.33
7	89	Electronics	167.15
8	98	Electronics	43.21
9	101	Electronics	99.44

```
Execution finished without errors.
Result: 52 rows returned in 5ms
At line 1:
-- Select product details
SELECT ProductID, Category, Price FROM Products WHERE Category = 'Electronics';
```

Goal : to retrieve the ProductID, Category, and Price of every item under the 'Electronics' category.
• **Expected Output**: A compilation of items falling under the 'Electronics' categorization. Every row in the output denotes a distinct product and includes information about its price, category ('Electronics'), and unique identification. This search aids in swiftly finding every electronic product and its cost.

```sql
-- Select customer information
SELECT Name, Age, MembershipLevel FROM Customers WHERE MembershipLevel = 'Gold';
```

	Name	Age	MembershipLevel
1	Ashley Baker	56	Gold
2	Stefanie Kramer	38	Gold
3	Gina Bradshaw	28	Gold
4	John Fox	41	Gold
5	Darryl Walters	50	Gold
6	Marcus Sanchez	66	Gold

```
Execution finished without errors.
Result: 193 rows returned in 5ms
At line 1:
-- Select customer information
SELECT Name, Age, MembershipLevel FROM Customers WHERE MembershipLevel = 'Gold';
```

• **Goal**: To obtain the membership levels, ages, and names of every customer who possesses a 'Gold' membership level.
• **Expected Output**: A table with the customer's name, age, and 'Gold' membership status. Targeting customers with 'Gold' status, this output can be used for marketing or customized customer service.

Joins queries :

```sql
-- Join Orders with Customers to find customer details for each order
SELECT o.OrderID, c.Name, c.MembershipLevel, o.OrderTotal
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID;
```

	OrderID	Name	MembershipLevel	OrderTotal
1	1	Alexandra Hester	Gold	838.28
2	2	William Castro	Bronze	149.14
3	3	Nancy Kramer	Gold	229.36
4	4	Sarah Rivera	Platinum	193.03
5	5	Lauren Bowers	Silver	246.64
6	6	Roger Rodriguez	Bronze	196.44

```
Execution finished without errors.
Result: 1000 rows returned in 93ms
At line 1:
-- Join Orders with Customers to find customer details for each order
SELECT o.OrderID, c.Name, c.MembershipLevel, o.OrderTotal
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID;
```

• **Goal**: By combining information from the Orders and Customers tables, this query seeks to present a comprehensive view of each order, complete with the customer's name, membership status, and order total.
• **Expected Output**: The ID of each order, the name of the customer who placed it, their membership level, and the order total are displayed in each row of the result set. Understanding consumer purchasing behavior and customizing marketing strategies require this information.

```sql
-- Join Orders with Products to list products details for each order
SELECT o.OrderID, p.Category, o.Quantity
FROM Orders o
JOIN Products p ON o.ProductID = p.ProductID;
```

	OrderID	Category	Quantity
1	1	Health and Beauty	2
2	2	Office Supplies	4
3	3	Health and Beauty	3
4	4	Automotive	3
5	5	Sports and Outdoors	2
6	6	Pet Supplies	3

```
Execution finished without errors.
Result: 1000 rows returned in 71ms
At line 1:
-- Join Orders with Products to list products details for each order
SELECT o.OrderID, p.Category, o.Quantity
FROM Orders o
JOIN Products p ON o.ProductID = p.ProductID;
```

The goal is to create a SQLite database schema with appropriate constraints and foreign key relationships to store data from retail stores, such as orders, customers, and products.
Expected Outcome: The database schema was successfully created, allowing for the effective management, retrieval, and storing of retail store data. This will also make future data analysis and operations easier.

Aggregations Queries:

```sql
-- Calculate the average order total by shipping method
SELECT ShippingMethod, AVG(OrderTotal) AS AvgOrderTotal
FROM Orders
GROUP BY ShippingMethod;
```

	ShippingMethod	AvgOrderTotal
1	Expedited	509.290881226054
2	Same-Day	517.636555555556
3	Standard	511.914
4	Two-Day	540.241741071428

```
Execution finished without errors.
Result: 4 rows returned in 8ms
At line 1:
-- Calculate the average order total by shipping method
SELECT ShippingMethod, AVG(OrderTotal) AS AvgOrderTotal
FROM Orders
GROUP BY ShippingMethod;
```

• **Goal**: The average total number of orders for each shipping option is to be determined here.
• **Expected Output**: A table containing the average order total for each shipping method will be generated. Understanding which shipping options are linked to orders of a higher value can be gained through this analysis, which may have an impact on the shipping choices made available to clients.

```sql
-- Find the total sales volume per product category
SELECT Category, SUM(SalesVolume) AS TotalSalesVolume
FROM Products
GROUP BY Category;
```

	Category	TotalSalesVolume
1	Automotive	184450
2	Baby and Toddler	166290
3	Clothing and Apparel	194513
4	Electronics	128386
5	Furniture	167998
6	Groceries	207719

```
Execution finished without errors.
Result: 15 rows returned in 9ms
At line 1:
-- Find the total sales volume per product category
SELECT Category, SUM(SalesVolume) AS TotalSalesVolume
FROM Products
GROUP BY Category;
```

• **Goal**: To compile the number of products sold in each category.
• **Expected Output**: An overview table with the overall sales volume for every product category listed. The categories that are most popular or drive the most sales are highlighted by this data, which is crucial for inventory planning and marketing.